VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Title of Project: ***Audit of Git – Distributed Version Control System***

Submitted by: *Sidharth Sharma*
Registration No. : *24BCE11452*

Contents of report

S.No.	Title	Page No.
1	Introduction	3
2	Part A- Origin and Philosophy	4-6
3	Part B- Linux Footprint	7-8
4	Part C- The FOSS Ecosystem	9
5	Part D — Open Source vs Proprietary (Critical Analysis)	10-11
6	Shell Scripts	12-19
7	Conclusion	20

Introduction

Open source software plays a vital role in today's digital world. Many of the systems and tools we use daily are built on open-source foundations. These systems are developed collaboratively and shared freely, allowing continuous improvement and innovation.

This project focuses on analyzing Git, a distributed version control system, to understand its origin, licensing, ecosystem, and its role in modern software development.

Part A — Origin and Philosophy

A1. Problem Addressed by Git

Git was developed in 2005 by Linus Torvalds to overcome the limitations of existing version control systems, particularly BitKeeper, which became proprietary and imposed restrictions on its use. This created significant challenges for collaborative software development, especially within the Linux kernel community.

Git addressed several critical issues, including slow development processes, limited support for distributed collaboration, and reliance on proprietary tools. By introducing a distributed version control system, Git enables each developer to maintain a complete local copy of the repository, thereby supporting offline work, improving performance, and enhancing collaboration efficiency.

A2. Licensing under GNU General Public License v2 (GPL v2)

Git is distributed under the GNU General Public License version 2 (GPL v2), which is a copyleft license designed to ensure software freedom. The license guarantees four essential freedoms: the freedom to run the program for

any purpose, to study and modify the source code, to redistribute copies, and to distribute modified versions.

While organizations are permitted to modify and commercially distribute Git or its derivatives, they are required to make the corresponding source code available and preserve the same license terms. This ensures that all derivative works remain open and accessible.

In contrast, permissive licenses such as the MIT License impose fewer restrictions, allowing proprietary use without the obligation to disclose source code. For projects requiring greater flexibility and fewer compliance requirements, the MIT License may be more suitable.

A3. Ethical Considerations in Open Source Software

Open source software plays a significant role in promoting collaboration, transparency, and innovation by enabling contributions from a global community of developers. It facilitates knowledge sharing and accelerates technological advancement.

However, certain ethical concerns arise, including the potential exploitation of unpaid contributors by commercial entities and the long-term sustainability of open source projects. Despite these challenges, Git serves as a

prominent example of how community-driven development can produce robust, widely adopted, and industry-defining technologies.

Part B — Linux Footprint

Git is installed on Linux systems using the Advanced Package Tool (APT) with the command `sudo apt install git`, indicating that it is available as a precompiled package and does not require manual compilation.

The primary binary is located at `/usr/bin/git`, while user-specific configuration files are stored in `~/.gitconfig`. Unlike many system applications, Git does not maintain log files in `/var/log` as it is not a daemon-based service.

Git runs under the current user rather than a dedicated system user or group. This is significant for security, as it ensures that repository operations are limited to the permissions of the active user, reducing the risk of unauthorized system-wide changes.

Git is not managed as a background service and therefore does not support service management commands such as start, stop, or restart via system managers like systemd.

Updates are handled through the package manager using commands such as `sudo apt update` and `sudo apt upgrade git`. Security patches and improvements are distributed by the open-source community through official repositories, ensuring regular and reliable updates.

Part C — The FOSS Ecosystem

Git operates within a broader open-source ecosystem and depends on standard system libraries and tools such as the GNU Compiler Collection (GCC) and libc for compilation and execution. It integrates with secure communication tools like OpenSSH for remote repository access.

Git has enabled and inspired numerous tools and platforms, including GitHub, GitLab, and Bitbucket, which provide collaboration, version control hosting, and DevOps functionalities built on top of Git.

Although Git is not a direct component of the LAMP stack, it plays a crucial role in web development by managing source code for web applications and enabling collaborative development workflows.

Git is supported by a large global community of developers, with contributions coordinated through mailing lists and repositories. Governance is primarily led by its creator, Linus Torvalds, along with a group of maintainers who review and integrate contributions.

Part D — Open Source vs Proprietary (Critical Analysis)

Comparison between chosen software(Git) against its closest proprietary alternative(Preforce).

Dimension	Git (Open Source)	Proprietary Alternative (Perforce)
Cost	Free of cost	Paid license required
Security (Auditability)	Source code is open and can be audited by anyone	Source code is closed; limited transparency

Support and Reliability	Community-driven support with wide documentation	Dedicated enterprise support with SLAs
Freedom to Modify	High; users can modify and distribute code	Limited; modification restricted by license
Community vs Corporate Control	Large global community; collaboratively governed	Controlled by a single company

Overall verdict: Git is better for most use cases

Shell Scripts

Script 1 — System Identity Report

Code:

```bash
#!/bin/bash
# Script 1: System Identity Report
# Author: Sidharth Sharma | Course: Open Source Software

STUDENT_NAME="Sidharth Sharma"
SOFTWARE_CHOICE="Git"

KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)
DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2 | tr -d '"')
HOME_DIR=$HOME

echo "================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "================================"
echo "Software : $SOFTWARE_CHOICE"
echo "Distro   : $DISTRO"
echo "Kernel   : $KERNEL"
echo "User     : $USER_NAME"
echo "Home Dir : $HOME_DIR"
echo "Uptime   : $UPTIME"
echo "Date     : $DATE"
echo "License  : GNU General Public License (GPL)"
```

Output:

```
sidharth@DESKTOP-SS:/mnt/c/Users/sharm/OneDrive/Desktop/COHORT NEW SERIES$ bash script1.sh
================================
 Open Source Audit — Sidharth Sharma
================================
Software : Git
Distro   : Ubuntu 24.04.4 LTS
Kernel   : 6.6.87.2-microsoft-standard-WSL2
User     : sidharth
Home Dir : /home/sidharth
Uptime   : up 13 minutes
Date     : Tue Mar 31 13:40:57 UTC 2026
License  : GNU General Public License (GPL)
```

This script generates a system identity report by displaying key information about the operating environment. It retrieves details such as the Linux distribution, kernel version, current user, home directory, system uptime, date, and license information. The script uses basic shell scripting concepts like variable declaration and command substitution (e.g., using commands like `uname`, `whoami`, `uptime`, and `date` to store outputs in variables). It also utilizes echo statements for formatted output and demonstrates simple scripting structure in Bash. Overall, the script provides a quick overview of system status using fundamental Linux commands and scripting techniques.

Script 2 — FOSS Package Inspector

Code:

```bash
#!/bin/bash
# Script 2: FOSS Package Inspector

PACKAGE="git"

if dpkg -l | grep -qw $PACKAGE; then
    echo "$PACKAGE is installed."
    dpkg -l $PACKAGE | grep "^ii"
else
    echo "$PACKAGE is NOT installed."
fi
case $PACKAGE in
    git) echo "Git: distributed version control system" ;;
    apache2) echo "Apache: web server powering the internet" ;;
    mysql) echo "MySQL: database system" ;;
    firefox) echo "Firefox: open source web browser" ;;
    *) echo "Unknown package" ;;
esac
```

Output:

```
sidharth@DESKTOP-SS:/mnt/c/Users/sharm/OneDrive/Desktop/COHORT NEW SERIES$ bash script2.sh
git is installed.
ii  git          1:2.43.0-1ubuntu7.3 amd64       fast, scalable, distributed revision control system
Git: distributed version control system
```

This script checks whether a specified software package is installed on the system and displays its details. It takes the package name as a command-line argument and uses the `dpkg -l` command along with `grep` to search for the package in the system's installed package list. If the package is found, it prints the installation status and related information; otherwise, it notifies that the package is not installed. The script also uses a case statement to provide a brief description of the software. Key shell scripting concepts used include conditional statements (if-else), command-line arguments, pattern matching with grep, and case control structures.

Script 3 — Disk and Permission Auditor
Code:

```bash
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist"
    fi
done
# Check Git config directory
echo ""
echo "Checking Git config directory..."

if [ -f "$HOME/.gitconfig" ]; then
    ls -l $HOME/.gitconfig
else
    echo "Git config file not found"
fi
```

Output:

```
sidharth@DESKTOP-SS:/mnt/c/Users/sharm/OneDrive/Desktop/COHORT NEW SERIES$  bash script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 4.3M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 17M
/home => Permissions: drwxr-xr-x root root | Size: 28K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 95M
/tmp => Permissions: drwxrwxrwt root root | Size: 24K

Checking Git config directory...
Git config file not found
```

This script performs a directory audit by analyzing important system directories and displaying their permissions and sizes. It checks directories such as `/etc`, `/var/log`, `/home`, `/usr/bin`, and `/tmp`, and retrieves their access permissions using the `ls -ld` command and their disk usage using the `du -sh` command. The script

uses a loop to iterate through a list of directories and conditionally verifies their existence before processing them. Additionally, it checks for the presence of the Git configuration file in the user's home directory. Key shell scripting concepts used include arrays, loops (for loop), conditional statements, command substitution, and basic file system commands.

Script 4 — Log File Analyzer

Code:

```
#!/bin/bash
# Script 4: Log File Analyzer

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File not found."
    exit 1
fi
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

# Show last 5 matching lines
echo ""
echo "Last 5 matches:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Output:

```
sidharth@DESKTOP-SS:/mnt/c/Users/sharm/OneDrive/Desktop/COHORT NEW SERIES$ bash script4.sh /var/log/syslog error
Keyword 'error' found 7 times in /var/log/syslog

Last 5 matches:
2026-03-31T13:28:01.273974+00:00 DESKTOP-SS kernel: RAS: Correctable Errors collector initialized.
2026-03-31T13:28:01.274451+00:00 DESKTOP-SS kernel: WSL (262) ERROR: CheckConnection: getaddrinfo() failed: -5
2026-03-31T13:28:01.515990+00:00 DESKTOP-SS snapd[208]: helpers.go:190: error trying to compare the snap system key: sys
tem-key missing on disk
2026-03-31T13:28:03.233560+00:00 DESKTOP-SS wsl-pro-service[212]: #033[33mWARNING#033[0m Could not ensure valid Landscap
e configuration: could not ensure valid Landscape configuration: could not register distro to Landscape: could not enabl
e Landscape: /usr/bin/landscape-config: error: exit status 1.
2026-03-31T13:39:13.853042+00:00 DESKTOP-SS wsl-pro-service[212]: #033[33mWARNING#033[0m Exiting after could not ensure
valid Landscape configuration: could not register distro to Landscape: could not enable Landscape: /usr/bin/landscape-co
nfig: error: exit status 1.
```

This script analyzes a system log file to count and display occurrences of a specific keyword such as “error” or “warning.” It accepts the log file path and keyword as command-line arguments, then processes the file line by line. The script uses text-processing tools like `grep` to identify matching lines and maintains a counter to track the total occurrences. Additionally, it displays the last few matching log entries for better insight into recent issues. Key shell scripting concepts used include command-line arguments, loops, conditional checks, counters, and text filtering using grep.

Script 5 — The Open Source Manifesto Generator

Code:

```bash
#!/bin/bash
# Script 5: Open Source Manifesto Generator

echo "Answer three questions to generate your manifesto."
echo ""

read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

echo "On $DATE, I believe in the power of open source." > $OUTPUT
echo "Using tools like $TOOL, I value $FREEDOM." >> $OUTPUT
echo "I aim to build and share $BUILD with the community." >> $OUTPUT

echo ""
echo "Manifesto saved to $OUTPUT"
cat $OUTPUT
```

Output:

```
sidharth@DESKTOP-SS:/mnt/c/Users/sharm/OneDrive/Desktop/COHORT NEW SERIES$  bash script5.sh
Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: git
2. In one word, what does 'freedom' mean to you? control
3. Name one thing you would build and share freely: a learning system

Manifesto saved to manifesto_sidharth.txt
On 31 March 2026, I believe in the power of open source.
Using tools like git, I value control.
I aim to build and share a learning system with the community.
```

This script generates a simple manifesto by taking user input interactively. It prompts the user with a series of questions related to open-source usage and collects responses using the read command. The script then combines these inputs into a formatted message and saves it into a text file. This demonstrates interactive shell scripting, file handling, and string manipulation. Key concepts used include user input handling (read), variables, output redirection, and basic script flow control.

Conclusion

Git is a highly flexible and cost-effective version control system suitable for most development environments. Its transparency, strong community support, and adaptability make it ideal for both individual developers and large-scale projects. However, proprietary tools like Perforce may offer advantages in enterprise environments requiring dedicated support and specialized features.

Overall, Git is recommended for real-world deployment in most scenarios due to its efficiency, scalability, and widespread adoption. Additionally, contributing to Git or its ecosystem is beneficial, as it supports continuous

improvement and strengthens the open-source community.